Gestión Reservas de Aulas

Óscar Reguera Lozano

Bases de Datos, Lenguaje de Marcas, Entornos de Desarrollo, Proyecto Intermodular y Programación--2º Evaluación, 1º DAW

25/02/2026

Contenido

3. Resumen Ejecutivo 3

4. Introducción 3

5. Análisis y diseño 4

Descripción del problema a resolver. 4

6. Desarrollo técnico 5

Descripción de las clases principales y sus responsabilidades. 5

7. Gestión de excepciones 7

Valoración de la robustez del programa. 7

8. Pruebas y validación 8

Casos de prueba realizados y resultados obtenidos. 8

9. Conclusiones y mejoras 8

Dificultades encontradas durante el desarrollo. 8

Soluciones adoptadas. 9

Propuestas de mejora (nuevas funcionalidades, refactorización, interfaz, etc.). 9

Competencias profesionales desarrolladas. 10

10. Bibliografía y webgrafía 10

11. Apéndices 10

Diagrama 10

Código 11

XML 20

3. Resumen Ejecutivo

El proyecto consiste en una aplicación Java de consola para la gestión de reservas de puestos y préstamos de dispositivos en centros educativos. Se ha implementado un diseño basado en Programación Orientada a Objetos que incluye la gestión de usuarios (alumnos y profesores), el control de disponibilidad de aulas mediante su capacidad, y la administración de material (portátiles y tablets). La solución integra persistencia en una base de datos relacional SQlite y la exportación de informes en formato XML consultables mediante XPath. El sistema automatiza además la detección de retrasos en las devoluciones para la aplicación de sanciones. Todo el desarrollo se ha gestionado con Git y se despliega sobre un servidor Ubuntu virtualizado.

4. Introducción

Este proyecto consiste en el diseño y desarrollo de una aplicación de consola en Java destinada a la gestión integral de centros de estudio. El sistema tiene como objetivo automatizar la reserva de puestos en aulas, controlando su disponibilidad según la capacidad de cada sala, y administrar el préstamo de dispositivos tecnológicos como portátiles y tablets. Mediante una interfaz de menús de texto, se facilita la interacción del usuario para realizar operaciones de alta, consulta y gestión de recursos en tiempo real.

La solución integra programación orientada a objetos con persistencia en bases de datos relacionales SQlite y el uso de archivos XML para la exportación de informes detallados.

Además, el software incorpora una lógica de control de fechas para detectar retrasos en las devoluciones y gestionar de forma automática las posibles sanciones a los usuarios.

5. Análisis y diseño

Descripción del problema a resolver.

El proyecto tiene como objetivo crear un sistema de gestión de aulas y equipos dentro de un centro educativo. El problema a resolver consiste en organizar y controlar de manera eficiente el uso de los recursos disponibles, evitando conflictos como la reserva de puestos ya ocupados o el préstamo de equipos que no estén disponibles.

Además, se busca mantener un registro de usuarios, aulas, puestos y equipos, facilitando la realización de reservas y préstamos de forma clara y segura. El sistema debe permitir que los administradores gestionen estos recursos de manera sencilla, garantizando que la información sea coherente y que las operaciones se realicen sin errores.

6. Desarrollo técnico

Descripción de las clases principales y sus responsabilidades.

1. Usuario

Clase base que representa a cualquier persona que utiliza el sistema. Contiene los atributos comunes id y nombre. Sirve como superclase para aplicar herencia y polimorfismo.

2. Alumno

Hereda de Usuario. Representa a un estudiante que puede realizar reservas de puestos y solicitar préstamos de equipos.

3. Profesor

Hereda de Usuario. Representa a un docente con las mismas capacidades básicas que un alumno, dejando abierta la posibilidad de añadir privilegios específicos.

4. Aula

Representa una sala o espacio físico. Contiene una lista de objetos Puesto, permitiendo organizar y gestionar los espacios disponibles.

5. Puesto

Representa un asiento o espacio individual dentro de un aula. Contiene un número identificador y un estado (ocupado o libre) para controlar la disponibilidad.

6. Equipo

Clase base que representa un dispositivo electrónico que puede prestarse. Contiene los atributos id y prestado para controlar su estado.

7. Portátil

Hereda de Equipo. Añade los atributos marca y modelo, permitiendo almacenar información específica de cada portátil.

8. Tablet

Hereda de Equipo. Incluye también los atributos marca y modelo, diferenciándose del portátil por el tipo de dispositivo.

9. Reserva

Representa la asociación entre un Usuario y un Puesto. Su función es registrar la reserva y marcar el puesto como ocupado.

10. Préstamo

Representa la relación entre un Usuario y un Equipo. Guarda la fecha del préstamo y permite gestionar la devolución del dispositivo.

11. GestorUsuarios

Se encarga de la gestión de usuarios: alta, listado y acceso a la colección de usuarios para su posterior exportación o gestión.

12. GestorReservas

Gestiona la creación de reservas, verificando que un puesto no esté ocupado antes de asignarlo a un usuario.

13. GestorPrestamos

Administra los préstamos de equipos, comprobando que el dispositivo esté disponible antes de realizar el préstamo.

14. GestorXMLUsuarios

Permite exportar la información de los usuarios a un archivo XML, cumpliendo el requisito de manejo de ficheros y facilitando futuras consultas mediante XPath.

15. App y Main

Constituyen el punto de entrada del programa. Inicializan los gestores y ejecutan la lógica principal del sistema.

7. Gestión de excepciones

Valoración de la robustez del programa.

El programa está bien estructurado y utiliza correctamente la programación orientada a objetos, aplicando herencia, encapsulamiento y separación en clases de gestión. Se controla que no se puedan reservar puestos ocupados ni prestar equipos no disponibles, y se comprueba que el usuario exista antes de realizar ciertas acciones, lo que aporta coherencia al funcionamiento.

No obstante, la robustez puede mejorarse.

8. Pruebas y validación

Casos de prueba realizados y resultados obtenidos.

```
PS C:\Users\oscar\Desktop\Proyecto Segunda evaluación Bueno>  & 'C:\Program Files\Java\jdk-25\bin\java.exe' '
--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\oscar\AppData\Roaming\Code\User\w
orkspaceStorage\beee5a5973cadd8d01590f14fec33177\redhat.java\jdt_ws\Proyecto Segunda evaluación Bueno_f9d0161
b\bin' 'Main'
Préstamo realizado.
Portátil ID: 101 | Marca: HP | Modelo: EliteBook 840 | Estado: Prestado
Tablet ID: 201 | Marca: Samsung | Modelo: Galaxy Tab S9 | Estado: Disponible
PS C:\Users\oscar\Desktop\Proyecto Segunda evaluación Bueno>
```

9. Conclusiones y mejoras

Dificultades encontradas durante el desarrollo.

Durante el desarrollo del programa se han encontrado varias dificultades. Una de las principales ha sido asegurar la coherencia entre las distintas clases y tipos de datos, evitando errores de incompatibilidad entre objetos, especialmente al trabajar con herencia y polimorfismo.

Otra dificultad ha sido gestionar correctamente los casos en los que los usuarios intentan realizar acciones no válidas, como reservar un puesto ya ocupado o prestar un equipo no

disponible, y garantizar que el sistema responda de forma segura sin generar errores en tiempo de ejecución.

Soluciones adoptadas.

Para resolver los problemas encontrados se tomaron varias medidas. Se comprobó que todas las clases y objetos funcionaran correctamente juntos, asegurando que las relaciones entre usuarios, equipos y reservas fueran coherentes.

Se añadieron controles para que no se pudiera reservar un puesto ocupado ni prestar un equipo ya en uso, y se verificó que el usuario existiera antes de realizar cualquier acción, evitando errores durante la ejecución.

Además, se organizó el programa en partes claras (gestores de usuarios, reservas y préstamos) y se añadieron comprobaciones básicas en el menú, lo que hace que el programa sea más seguro, fácil de usar y de mantener.

Propuestas de mejora (nuevas funcionalidades, refactorización, interfaz, etc.).

Como propuestas de mejora, sería recomendable reforzar el control de errores, incorporando gestión de excepciones para evitar fallos si el usuario introduce datos incorrectos en el menú. También se podría añadir una validación más completa de los datos, como evitar usuarios duplicados o limitar el número de reservas y préstamos por persona.

Competencias profesionales desarrolladas.

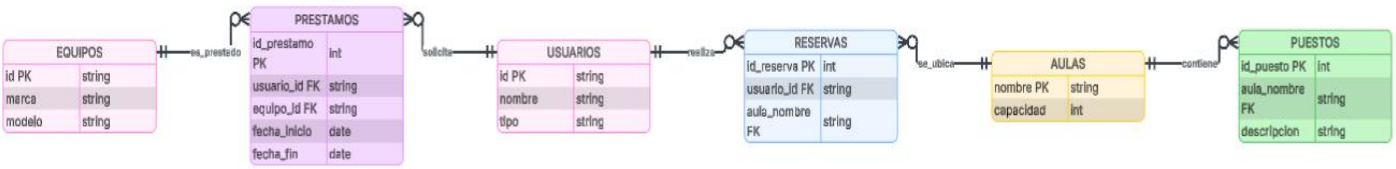
Con este programa se han desarrollado competencias relacionadas con la programación orientada a objetos, aplicando conceptos como herencia, encapsulamiento y polimorfismo para organizar correctamente el sistema. También se ha trabajado la capacidad de análisis y diseño, dividiendo el problema en distintas clases y gestionando la información mediante listas.

10. Bibliografía y webgrafía

https://www.tutorialesprogramacionya.com/

11. Apéndices

Diagrama



Código

```
public class Alumno extends Usuario {

    public Alumno(int id, String nombre) {
        super(id, nombre);
    }
}
```

```java
public  class App {

    public static void iniciar() {

        GestorPrestamos gestorPrestamos = new GestorPrestamos();

        Alumno alumno = new Alumno(id: 1, nombre: "Carlos");

        Portatil portatil = new Portatil(id: 101, marca: "HP", modelo: "EliteBook 840");
        Tablet tablet = new Tablet(id: 201, marca: "Samsung", modelo: "Galaxy Tab S9");

        gestorPrestamos.hacerPrestamo(alumno, portatil);
    ;       You, 48 minutes ago • proyecto …      Empty statement
                                                  Cerrar
        System.out.println(portatil);
        System.out.println(tablet);
    }


}
```

```java
import java.util.ArrayList;

You, 49 minutes ago | 1 author (You)
public class Aula {
    private String nombre;     Field nombre can be final
    private ArrayList<Puesto> puestos;     Field puestos can be final

    public Aula(String nombre) {
        this.nombre = nombre;
        this.puestos = new ArrayList<>();
    }

    public void añadirPuesto(Puesto p) {
        puestos.add(p);                                  Cerrar
    }

    public String getNombre() { return nombre; }
    public ArrayList<Puesto> getPuestos() { return puestos; }
}
```

```java
public class Equipo {
    protected int id;
    protected boolean prestado;

    public Equipo(int id) {
        this.id = id;
        this.prestado = false;
    }

    public int getId() {
        return id;
    }

    public boolean isPrestado() {
        return prestado;
    }

    public void prestar() {
        prestado = true;
    }

    public void devolver() {
        prestado = false;
    }
}
```

```java
import java.util.ArrayList;

public class GestorPrestamos {
    private ArrayList<Prestamo> prestamos = new ArrayList<>();

    public void hacerPrestamo(Usuario u, Equipo e) {
        if (!e.isPrestado()) {
            prestamos.add(new Prestamo(u, e));
            System.out.println("Préstamo realizado.");
        } else {
            System.out.println("Equipo ya prestado.");
        }
    }
}
```

```java
import java.util.ArrayList;

You, 50 minutes ago | 1 author (You)
public class GestorReservas {
    private ArrayList<Reserva> reservas = new ArrayList<>();

    public void hacerReserva(Usuario u, Puesto p) {
        if (!p.isOcupado()) {
            reservas.add(new Reserva(u, p));
            System.out.println(x: "Reserva realizada.");
        } else {
            System.out.println(x: "El puesto ya está ocupado.");
        }
    }
}      You, 50 minutes ago • proyecto …
```

Cerrar

```java
import java.util.ArrayList;

public class GestorUsuarios {
    private ArrayList<Usuario> usuarios = new ArrayList<>();

    public void añadirUsuario(Usuario u) {
        usuarios.add(u);
    }

    public void listarUsuarios() {
        for (Usuario u : usuarios) {
            System.out.println(u);
        }
    }
}
```

Cerrar

```java
...
public class Main {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        App.iniciar();
    }

}
```

```java
public class Portatil extends Equipo {

    private String marca;    Field marca can be final
    private String modelo;    Field modelo can be final

    public Portatil(int id, String marca, String modelo) {
        super(id);
        this.marca = marca;
        this.modelo = modelo;
    }

    public String getMarca() {
        return marca;
    }                                                    Cerrar

    public String getModelo() {
        return modelo;
    }

    @Override
    public String toString() {
        return "Portátil ID: " + id +
                " | Marca: " + marca +
                " | Modelo: " + modelo +
                " | Estado: " + (prestado ? "Prestado" : "Disponible");
    }
```

```java
import java.time.LocalDate;

You, 52 minutes ago | 1 author (You)
public class Prestamo {
    private Usuario usuario;    Field usuario can be final
    private Equipo equipo;    Field equipo can be final
    private LocalDate fechaPrestamo;    Field fechaPrestamo can be final

    public Prestamo(Usuario usuario, Equipo equipo) {
        this.usuario = usuario;
        this.equipo = equipo;
        this.fechaPrestamo = LocalDate.now();
        equipo.prestar();
    }                                                    Cerrar

    public void devolver() {
        equipo.devolver();
    }

    @Override
    public String toString() {
        return "Préstamo de " + usuario.getNombre() + " - Fecha: " + fechaPrestamo;
    }
}       You, 52 minutes ago • proyecto
```

```java
public class Profesor extends Usuario {

    public Profesor(int id, String nombre) {
        super(id, nombre);
    }
}       You, 52 minutes ago • proyecto …
```

```java
public class Puesto {
    private int numero;    Field numero can be final
    private boolean ocupado;

    public Puesto(int numero) {
        this.numero = numero;
        this.ocupado = false;
    }

    public boolean isOcupado() { return ocupado; }
    public void ocupar() { ocupado = true; }
    public void liberar() { ocupado = false; }

    @Override                                                    Cerrar
    public String toString() {
        return "Puesto " + numero + " - " + (ocupado ? "Ocupado" : "Libre");
    }
}       You, 53 minutes ago • proyecto …
```

```java
public class Reserva {
    private Usuario usuario;    Field usuario can be final
    private Puesto puesto;    Field puesto can be final

    public Reserva(Usuario usuario, Puesto puesto) {
        this.usuario = usuario;
        this.puesto = puesto;
        puesto.ocupar();
    }

    @Override
    public String toString() {
        return "Reserva de " + usuario.getNombre();
    }
}
```

```java
public class Tablet extends Equipo {

    private String marca;    Field marca can be final
    private String modelo;    Field modelo can be final

    public Tablet(int id, String marca, String modelo) {
        super(id);
        this.marca = marca;
        this.modelo = modelo;
    }

    public String getMarca() {
        return marca;
    }

    public String getModelo() {
        return modelo;
    }

    @Override
    public String toString() {
        return "Tablet ID: " + id +
                " | Marca: " + marca +
                " | Modelo: " + modelo +
                " | Estado: " + (prestado ? "Prestado" : "Disponible");
    }
}
```

```java
public class Usuario {
    protected int id;
    protected String nombre;

    public Usuario(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

    public int getId() { return id; }
    public String getNombre() { return nombre; }

    @Override
    public String toString() {
        return id + " - " + nombre;
    }
}
```

Cerrar

You, 53 minutes ago • proyecto …

XML

```java
import java.io.File;
import java.util.List;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.Document;
import org.w3c.dom.Element;

You, 54 minutes ago | 1 author (You)
public class XmlUsuarios {

    public static void guardarUsuariosXML(List<Usuario> usuarios) {

        try {

            DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
            DocumentBuilder builder = factory.newDocumentBuilder();
            Document doc = builder.newDocument();

            Element raiz = doc.createElement(tagName: "usuarios");
            doc.appendChild(raiz);

            for (Usuario u : usuarios) {

                Element usuario = doc.createElement(tagName: "usuario");
```

```java
                if (u instanceof Alumno) {
                    usuario.setAttribute(name: "tipo", value: "Alumno");
                } else if (u instanceof Profesor) {
                    usuario.setAttribute(name: "tipo", value: "Profesor");
                }

                Element id = doc.createElement(tagName: "id");
                id.appendChild(doc.createTextNode(String.valueOf(u.getId())));    Dereferencing possible n
                usuario.appendChild(id);

                Element nombre = doc.createElement(tagName: "nombre");
                nombre.appendChild(doc.createTextNode(u.getNombre()));
                usuario.appendChild(nombre);

                raiz.appendChild(usuario);
            }

            TransformerFactory transformerFactory = TransformerFactory.newInstance();
            Transformer transformer = transformerFactory.newTransformer();

            transformer.setOutputProperty(OutputKeys.INDENT, value: "yes");
            transformer.setOutputProperty(name: "{http://xml.apache.org/xslt}indent-amount", value: "4");

            DOMSource source = new DOMSource(doc);
        File archivo = new File(pathname: "usuarios.xml");
System.out.println("Se guardará en: " + archivo.getAbsolutePath());
```

```
System.out.println("Se guardará en: " + archivo.getAbsolutePath());

StreamResult result = new StreamResult(archivo);

            transformer.transform(source, result);

            System.out.println(x: "XML generado correctamente.");

        } catch (Exception e) {     Can be replaced with multicatch or
            e.printStackTrace();     Print Stack Trace
        }                                                   Cerrar
    }
}       You, 54 minutes ago • proyecto
```